

Rilevazione e predizione di grandezze fisiche ambientali tramite un sistema IoT

Merlo Pietro^{1*}, Emanuele Lattanzi²

Sommario

Negli ultimi anni il cambiamento climatico ha reso sempre più importante il monitoraggio continuo e costante dei parametri ambientali. In questo contesto, l'Internet of Things riveste un ruolo sempre più rilevante, poiché consente di realizzare sistemi distribuiti in grado di acquisire, trasmettere e analizzare dati in modo economico ed efficiente. In questo progetto è stato realizzato un sistema IoT basato su ESP32 e Raspberry Pi per la raccolta, l'elaborazione e la memorizzazione di misurazioni di temperatura, luminosità e umidità dell'aria. Viene utilizzato il protocollo MQTT per lo scambio di informazioni tra i due dispositivi mentre, tramite delle API meteorologiche, è possibile acquisire misurazioni esterne; i dati acquisiti vengono inviati ad un'istanza cloud di InfluxDB. Tali informazioni costituiranno poi la base per le successive analisi statistiche e per l'applicazione di modelli di Machine Learning.

Keywords

Internet of Things — Machine Learning — Raspberry Pi — ESP32 — MQTT — InfluxDB — API

1Laurea Magistrale in Informatica Applicata, Università degli Studi di Urbino Carlo Bo, Urbino, Italia

2Docente di Programmazione per l'Internet of Things, Università degli Studi di Urbino Carlo Bo, Urbino, Italia

*Corresponding author: p.merlo@campus.uniurb.it

1. Introduzione

Negli ultimi decenni, il cambiamento climatico ha reso sempre più necessaria l'osservazione continua dei fenomeni ambientali e la possibilità di disporre di dati affidabili su cui basare analisi e decisioni. In questo contesto l'Internet of Things ha assunto un ruolo sempre più rilevante poiché consente di collocare sul territorio una serie di sensori e dispositivi intelligenti capaci di raccogliere informazioni rilevanti in modo automatico e continuativo.

Uno degli aspetti che rende l'IoT particolarmente interessante in ambito ambientale è il rapporto molto vantaggioso che esso possiede tra costi, accessibilità ed efficacia. La diffusione di sensori sempre più economici, di microcontrollori sempre più potenti e a basso consumo energetico permette, ad oggi, di realizzare sistemi di monitoraggio con investimenti contenuti facendo sì che queste tecnologie siano diventate utilizzabili non solo in ambito industriale ma anche in contesti sperimentali, didattici o di ricerca.

Allo stesso tempo, gli stessi fattori che rendono l'IoT particolarmente attraente, possono porre anche alcune criticità, legate al consumo energetico, alla gestione della grande quantità di dati prodotta e alla necessità di progettare infrastrutture e soluzioni affidabili e sostenibili nel tempo. È proprio per questi motivi che l'interesse nei confronti dell'IoT è cresciuto fortemente negli ultimi anni: da un lato offre strumenti concreti per osservare e comprendere meglio l'ambiente che ci circonda, dall'altro richiede particolare attenzione verso l'efficienza e l'ottimizzazione dei sistemi che propone.

L'obiettivo del presente progetto è la realizzazione di un sistema IoT distribuito per il monitoraggio di alcune grandezze fisiche ambientali come temperatura, umidità e luminosità. Per raggiungere tale scopo sono stati utilizzati due dispositivi noti in campo IoT come ESP32 e Raspberry Pi la cui mansione è la raccolta, l'elaborazione e la successiva analisi dei dati raccolti.

2. Progettazione del sistema

Il sistema realizzato è stato progettato seguendo un'architettura tipica dei sistemi a livello edge, basata quindi su componenti distribuiti che operano il più vicino possibile alla sorgente del dato. In questo modello i dispositivi possono essere distinti in due categorie principali: le *smart things*, che possiedono sensori, attuatori e una contenuta capacità di elaborazione locale, e i *gateways*, i quali forniscono connettività tra i dispositivi locali e la rete internet, fornendo a loro volta sensoristica e capacità di processing più avanzata. Seguendo questo paradigma, il prototipo sviluppato è composto da un nodo periferico costituito da una ESP32, incaricata dell'acquisizione delle grandezze ambientali richieste, e da un nodo centrale, ricoperto da una Raspberry Pi, che svolge a sua volta funzioni di raccolta, integrazione e gestione dei dati. Questa suddivisione riproduce, seppur su piccola scala, una tipica struttura IoT distribuita, nella quale nodi periferici e nodi di coordinamento comunicano e cooperano tra di loro per completare il loro scopo.

Il primo modulo di questo sistema è basato su una scheda ESP32 con DevKit, la quale, all'interno del sistema ricopre il ruolo di *smart thing*. Questo dispositivo è infatti incaricato della rilevazione locale delle grandezze ambientali richieste e rappresenta dunque il primo nodo periferico di sensing dell'architettura. Nel dettaglio, l'ESP32, il cui schema elettronico è riportato in figura [1], acquisisce i valori di temperatura e umidità dell'aria tramite un sensore *DHT11* collegato al pin GPIO25. In aggiunta, è stato collegato un LED di segnalazione, collegato al pin GPIO32, utilizzato come indicatore visivo dell'attività del nodo durante l'esecuzione delle operazioni di misura e trasmissione dei dati. L'alimentazione di questo modulo proviene dal micro-connettore USB, il quale, ai fini del progetto è stato collegato alla Raspberry Pi ma che, nell'ottica di un sistema maggiormente distribuito, può essere collegato a sorgenti energetiche come batterie o powerbank, rendendo questo modulo totalmente indipendente.

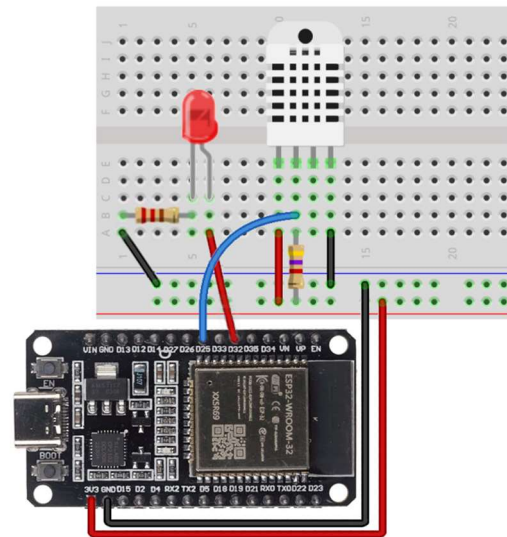


Figura 1. Schematica ESP32 con LED e DHT11

Il secondo modulo di questo sistema è costituito da una Raspberry Pi 5 che svolge il ruolo di *gateway*. Nella configurazione utilizzata per lo svolgimento di questo progetto, il gateway non si limita a ricevere i dati dal nodo periferico ma fornisce ulteriori funzionalità di sensing, integrazione dei dati e computazione, assumendo un ruolo di elevata importanza all'interno del sistema. Per quanto riguarda la configurazione hardware di questo modulo, riportato in figura [2], alla Raspberry Pi è collegato un convertitore analogico-digitale MCP3008 e un ulteriore LED, sempre con funzione di segnalazione, al pin GPIO4. Il convertitore MCP3008 misura la luminosità dell'ambiente tramite una fotoresistenza, i cui valori vengono letti sul canale 2 e comunicati alla Raspberry Pi tramite protocollo SPI. Per un funzionamento stabile e sicuro di questo sistema, questo modulo è stato alimentato tramite una presa a muro che fornisce una tensione stabile ai dispositivi.

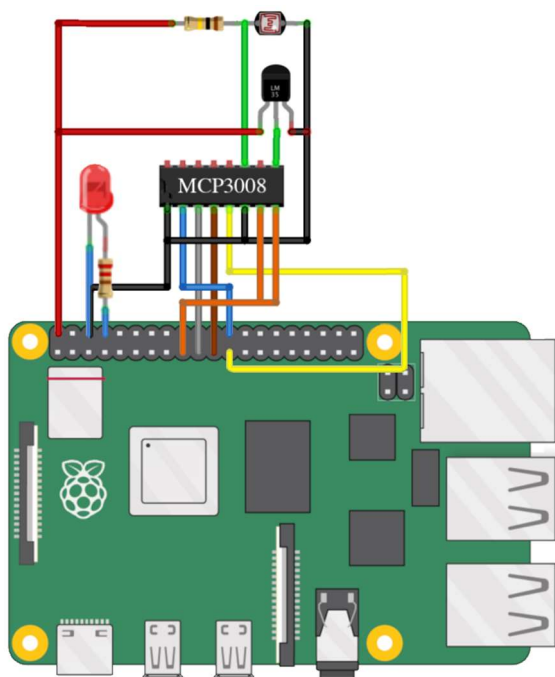


Figura 2. Schematica Raspberry Pi 5 con LED e MCP3008

A supporto dei due moduli principali, è stato aggiunto un piccolo sistema di ventilazione, denominato *WALL-E*, mostrato in figura [3]. La sua presenza è utile a mantenere un flusso d'aria costante in prossimità dei dispositivi, riducendo l'influenza dell'eventuale surriscaldamento dei dispositivi stessi sulle misurazioni effettuate. La sua alimentazione proviene da una delle porte USB di Raspberry Pi.



Figura 3. Sistema di ventilazione WALL-E

3. Implementazione e flusso operativo

In questo capitolo verranno analizzati i codici prodotti e utilizzati in questo progetto, analizzandone le caratteristiche principali per metterne in evidenza il comportamento dei due moduli.

3.1 ESP32

Il nodo periferico, costituito dalla ESP32, è stato programmato in ambiente Arduino ed è responsabile della prima acquisizione delle misure ambientali. In particolare, dopo l'avvio, il dispositivo esegue una fase di setup in cui inizializza le costanti per la connessione Wi-Fi

```
char WIFI_SSID[] = "TIM FWA-G3R7";
char WIFI_PASSWORD[] = "*****";
```

e le costanti per la configurazione di MQTT, utilizzando l'IP della Raspberry Pi come broker.

```
const char broker[] = "192.168.1.111";
int port = 1883;
const char topic[] = "sensori_ESP_merlo503";
char clientID[] = "esp99";
```

Successivamente, crea un'istanza del sensore DHT11 e definisce il pin utilizzato dal led.

```
#define DHTPIN 25
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);
const int led = 32;
```

A questo punto entra nella funzione *setup()*, all'interno della quale sono contenute le istruzioni che ne definiscono il vero comportamento: dopo aver inizializzato la comunicazione seriale e aver attivato il sensore DHT11, la ESP32 tenta la connessione alla rete Wi-Fi definita in precedenza per 15 secondi e in caso di mancata connessione, entra in una fase di *deepsleep* per poi ritentare il tutto al suo risveglio. Se la connessione Wi-Fi ha successo, tenta la connessione al broker MQTT con

```
mqttClient.connect(broker,port)
```

e, analogamente al tentativo di connessione Wi-Fi, in caso di mancata connessione, entra in una fase di *deepsleep*. Se la connessione al broker MQTT, ruolo ricoperto in questo progetto da Raspberry Pi, avviene con successo, inizia la fase di misurazione delle grandezze ed invio dei dati: vengono lette le misure di temperatura ed umidità tramite DHT11

```
tempC = dht.readTemperature();
hum = dht.readHumidity();
```

e con le misure appena ottenute viene costruita una stringa compatta, inviata sul topic definito in precedenza con

```
mqttClient.beginMessage(topic, false, 1);
mqttClient.print(message);
mqttClient.endMessage();
```

e infine, viene fatto lampeggiare brevemente il led come indicatore visivo del completamento dell'invio dei dati a Raspberry Pi.

A questo punto, la ESP32 entra nella fase di *deepsleep*: per puntualizzare, analogamente ai casi di mancata connessione Wi-Fi / MQTT, prima di entrare nel *deepsleep* viene calcolato il tempo trascorso dal momento dal risveglio al momento attuale. In questo modo, la fase di sonno del dispositivo sarà di lunghezza complementare al tempo utilizzato per lo svolgimento dei suoi compiti e così facendo, si garantisce che le misurazioni avvengano ad intervalli sufficientemente regolari, di 5 minuti. L'utilizzo del *deepsleep* è stato implementato seguendo il paradigma IoT di utilizzare dispositivi a basso fabbisogno energetico e di ottimizzarne il consumo. Ai fini di questo progetto, i due dispositivi devono stare vicini tra di loro per garantire la congruenza spaziale dei dati che raccolgono e di conseguenza è stato deciso di alimentare ESP32 collegandola a Raspberry Pi. L'utilizzo del *deepsleep*, tuttavia, rende possibile un eventuale utilizzo autonomo di questo dispositivo il quale, avendo un basso consumo energetico, può essere alimentato a batteria o tramite powerbank e può quindi essere posizionato ovunque, a patto che riesca comunque a connettersi alla rete Wi-Fi.

3.2 Raspberry Pi

Raspberry Pi 5, in questo progetto, costituisce il nodo centrale del sistema e svolge una funzione di gateway, integrazione dati e invio delle informazioni ad un database remoto. Il codice è stato sviluppato in Python e a differenza di ESP32, vista la natura del ruolo di questo dispositivo, rimane in esecuzione e in attesa di messaggi ininterrottamente. Prima di descrivere il codice, è opportuno puntualizzare che su questo nodo è stato installato e configurato il servizio di broker MQTT Mosquitto. L'installazione di Mosquitto e delle sue utility è avvenuta tramite linea di comando con

```
sudo apt update
sudo apt install -y mosquitto mosquitto-clients
```

Sempre tramite linea di comando, è stato modificato il file di configurazione

```
sudo nano /etc/mosquitto/mosquitto.conf
```

abilitando l'ascolto sulla porta standard e consentendo l'accesso anonimo, aggiungendo al file di configurazione le righe

```
listener 1883
allow_anonymous true
```

Infine, per fare in modo che Mosquitto si avvii automaticamente ad ogni accensione di Raspberry Pi e per avviarlo immediatamente si utilizza l'interfaccia *systemctl* come segue:

```
sudo systemctl enable mosquitto
sudo systemctl start mosquitto
```

Per quanto riguarda il codice, invece, come accennato in precedenza, esso rimane in esecuzione permanente. Inizialmente sono definiti i parametri per la connessione MQTT locale

```
MQTT_BROKER = "localhost"
MQTT_PORT = 1883
MQTT_TOPIC = "sensori_ESP_merlo503"
```

e i parametri per l'invio dei dati ad InfluxDB.

```
myToken="---token---"
myOrg="studenteUniurb"
myBucket="corsoIoT"
myHost="ProgettoN503"
myUrl="http://informatica-iot.freedomdns.org:8086/"
```

Vengono poi create le istanze per l'utilizzo del led e del convertitore analogico-digitale MCP3008, leggendone il valore relativo alla fotoresistenza tramite il canale 2.

```
led=LED(4)
lux_reader = MCP3008(channel=2,
    differential=False, max_voltage=1.5)
```

Dopo aver creato l'istanza per l'invio dei dati ad InfluxDB e aver configurato le API di scrittura con

```
client = InfluxDBClient(url=myUrl,
    token=myToken, org=myOrg)
write_api=client.write_api(
    write_options=SYNCHRONOUS)
```

vengono definite le tre funzioni principali di questa implementazione: la funzione *get_real_data()* ottiene i dati relativi a temperatura ed umidità reali tramite una richiesta http alle API di Open-Meteo.

La funzione `on_connect()` effettua solamente la sottoscrizione al topic MQTT definito precedentemente con

```
client.subscribe(MQTT_TOPIC, qos=1)
```

mentre all'interno della funzione `on_message()` è contenuto il vero comportamento del dispositivo: quando viene ricevuto un messaggio tramite protocollo MQTT, inizialmente viene fatto lampeggiare il led per segnalare l'evento. Viene effettuato il *parsing* della stringa compatta ricevuta da ESP32, viene misurata la luminosità tramite il comando

```
lux = round(lux_reader.value*100,1)
```

e viene invocata la funzione `get_real_data()`, descritta precedentemente, per ottenere le misurazioni provenienti dall'esterno. A questo punto, dopo aver creato i cinque *point*, uno per ogni misura, aggiungendo ai valori ottenuti i relativi tag, questi vengono inviati ad InfluxDB con

```
write_api.write(bucket=myBucket,
    record =[point1, point2, point3, point4,
    point5])
```

Infine, dopo aver definito queste funzioni, è presente il codice effettivamente eseguito dal dispositivo che si limita a connettersi al broker MQTT e a restare per sempre in attesa di messaggi con

```
client_mqtt = mqtt.Client()
client_mqtt.on_connect = on_connect
client_mqtt.on_message = on_message
client_mqtt.connect(MQTT_BROKER, MQTT_PORT, 60)
client_mqtt.loop_forever()
```

4. Raccolta e analisi dei dati

La fase di raccolta dati è stata svolta acquisendo periodicamente le misure di temperatura, umidità e luminosità tramite i sensori dei due dispositivi e tramite interrogazione delle API di Open-Meteo. I dati raccolti coprono un periodo di circa due settimane, dal 19 Giugno 2026 al 3 Luglio 2026 e sono stati campionati ogni 5 minuti ottenendo circa 3400 campioni. La stanza individuata per la raccolta si trova al piano terra dell'abitazione in cui ho vissuto in tale periodo ed è stata scelta perché poco utilizzata. Proprio grazie al suo utilizzo limitato è stato possibile mantenere costanti le condizioni che avrebbero potuto influenzare le misurazioni dei dati come tapparelle

alzate/abbassate, finestre aperte/chiuso, luci artificiali accese/spente ed eventuali sistemi di riscaldamento/raffreddamento. I dispositivi sono stati posizionati vicino ad una finestra chiusa con tapparella alzata facendo in modo che i sensori non venissero mai colpiti direttamente dalla luce del sole. Le luci della stanza sono state lasciate perlopiù spente e non sono mai stati accesi termosifoni o condizionatori

4.1 Instabilità della connessione Wi-Fi

Per la connessione Wi-Fi dei due dispositivi presenti in questo progetto è stato utilizzato un modem apposito posizionato nella stanza della raccolta dei dati. Durante il periodo di acquisizione, tuttavia, per motivi non del tutto noti ma probabilmente a causa dell'instabilità della connessione, ci sono stati dei periodi, mostrati in figura [4] in cui non è stata possibile la comunicazione tramite protocollo MQTT tra i due dispositivi né tantomeno l'invio delle misurazioni ad InfluxDB.



Figura 4. Parti mancanti nella serie temporale

Le parti marcate in rosso in figura [4] costituiscono circa il 14% del periodo totale di rilevazione. La soluzione trovata per ripristinare il normale flusso operativo del sistema è stato un semplice riavvio forzato del modem ma purtroppo non è sempre stato possibile effettuarlo tempestivamente.

4.2 Instabilità delle richieste API

Una volta terminata la fase di raccolta dei dati, è stata notata, inoltre, una piccola discrepanza tra il numero di misurazioni totali ottenute dai sensori, 3402 campioni, e il numero totale di misurazioni ottenute tramite richiesta API, 3381. Per ovviare a questa problematica, e a quella citata al punto 4.1, è stato utilizzato uno script python che, a partire dai file .CSV contenenti le cinque misurazioni, ha creato un unico file .CSV contenente solamente righe "complete", ovvero in cui sono presenti tutti i valori per ogni timestamp.

4.3 Analisi dei dati

Nelle tabelle [1] e [2] sono riportate le misure di media, deviazione standard, massimo e minimo relative ai periodi *giorno* e *notte*, la divisione è stata effettuata in base ai valori di luminosità misurati. Nello specifico è stata definita *notte* il periodo in cui la luminosità è compresa tra 90 e 100 e *giorno* il periodo restante.

Giorno	media	std dev	min	max
Lux	38.8	20.28	2.0	89.9
Temp	27.5	0.97	25.2	29.3
API Temp	29.9	3.87	18.7	36.7
Hum	67.0	11.32	48	95
API Hum	51.9	14.85	30	91

Tabella 1. Metriche relative al periodo *giorno*

Notte	media	std dev	min	max
Lux	99.7	1.29	90.1	100.0
Temp	27.4	1.01	25.2	29.2
API Temp	24.9	2.80	18.7	32.6
Hum	66.7	11.14	46	92
API Hum	65.5	13.72	34	92

Tabella 2. Metriche relative al periodo *notte*

I dati mostrati in tabella [1] e tabella [2] mostrano come cambino i valori delle varie grandezze misurate tra giorno e notte. In particolare, possiamo notare come la temperatura media misurata all'interno della stanza non subisca variazioni particolari per quanto riguarda il suo valor medio, 27.5 °C di notte rispetto a 27.4 °C di giorno, mentre all'esterno è presente una differenza più marcata. La temperatura media misurata nei due intervalli mostra un calo dai 29.9 °C diurni ai 24.9 °C notturni. Anche l'umidità interna ha un comportamento simile: non mostra infatti notevoli variazioni tra i valori medi misurati di giorno e di notte. Le misure esterne invece, mostrano un comportamento opposto rispetto alla controparte relativa alla temperatura. Notiamo infatti che l'umidità esterna media sale da 51.9% a 65.5% tra giorno e notte. Possiamo notare inoltre come sebbene durante il giorno, la temperatura interna sia stata mediamente inferiore a quella esterna di circa 2.5 °C, durante la notte accade il contrario, la temperatura esterna media risulta minore, sempre di circa 2.5 °C. Un'ulteriore valutazione può essere fatta relativamente all'escursione termica: durante il giorno, ad esempio, la temperatura esterna va da un

minimo di 18.7 °C ad un massimo di 36.7 °C mentre quella interna oscilla tra 25.2 °C e 29.3 °C. Lo stesso può essere visto durante la notte. Queste misure mostrano una forte inerzia termica dell'ambiente interno che sicuramente aiuta a proteggere dalle alte temperature raggiunte durante il giorno ma allo stesso tempo non permette alla temperatura interna di scendere molto velocemente durante la notte, restando mediamente più alta rispetto a quella esterna. Infine, durante il periodo di raccolta dei dati, il continente europeo è stato colpito da una forte ondata di calore, da notare in particolare il valore massimo raccolto di temperatura esterna: il giorno 29 Giugno 2026, alle 13:35, a Marano Vicentino sono stati infatti misurati 36.7°C. Differentemente da ciò che ci si può aspettare, il valore massimo di temperatura interna, 29.3 °C, non è stato misurato contemporaneamente o successivamente a tale misurazione ma bensì qualche giorno prima, il 23 Giugno 2026 alle 16:55.

In figura [5] sono riportati gli indici di correlazione di Pearson per ogni misura rilevata, in forma matriciale. Il coefficiente di Pearson è una misura che quantifica il grado di associazione lineare tra due variabili: il suo valore va da -1 a +1, valori positivi indicano una relazione diretta, valori negativi indicano una relazione inversa mentre valori prossimi allo zero indicano una relazione lineare assente o molto debole.

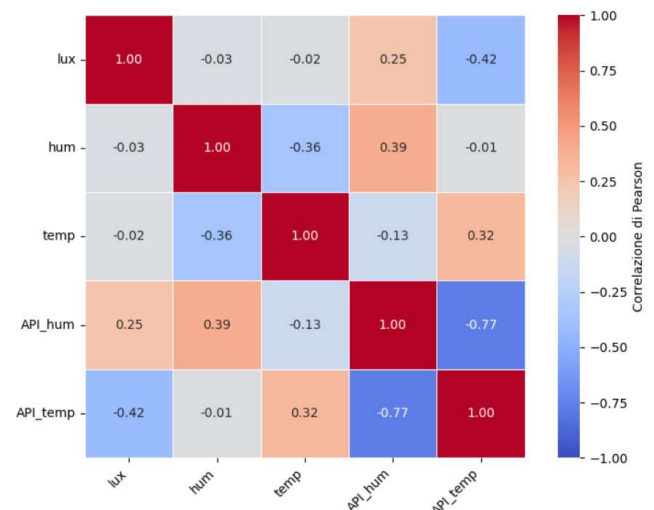


Figura 5. Matrice di correlazione di Pearson

Dalla matrice di correlazione in figura [4] si evince che la luminosità non mostra relazioni lineari forti rispetto alle altre grandezze misurate all'interno. Mostra invece coefficienti non banali in relazione a temperatura e umidità esterne, rispettivamente 0.25 e -0.42. Sempre considerando le grandezze misurate all'interno, appare evidente una correlazione negativa, seppur moderata, tra temperatura e umidità, pari a -

0.36: ciò significa che all'aumentare della temperatura misurata all'interno della stanza, l'umidità tenderà a diminuire e viceversa. Come da pronostico, è presente un fattore di correlazione positivo tra le misure di umidità e temperatura interne e le rispettive controparti esterne: ciò lascia intendere che l'ambiente domestico risente delle condizioni esterne ma in misura limitata, visti i coefficienti piuttosto bassi, evidenziando ancora una volta la capacità delle abitazioni di smorzare le variazioni climatiche provenienti dall'esterno. Infine, tra temperatura e umidità esterne, si osserva il coefficiente di entità maggiore, pari a -0.77 , evidenziando il rapporto inverso e piuttosto marcato tra le due grandezze.

4.4 Stima dei consumi ESP32

Successivamente alla fase di raccolta dati è stato svolto un breve esperimento per stimare il consumo di ESP32 durante la fase di acquisizione dei dati. È stata aumentata la frequenza di acquisizione delle misure in modo che il dispositivo effettuasse le misurazioni ogni minuto, non è stato cambiato in nessun modo il comportamento dei due moduli ma è stata inviata ad influxDB solamente la quantità di tempo impiegata da ESP32 per raccogliere i dati e inviarli a Raspberry Pi. Sono stati raccolti 877 campioni e sono state calcolate le misure riportate nella tabella [3].

ESP32	media	std dev	min	max
Uptime	2.08	0.52	1.6	11.5

Tabella 3. Misure fase attiva ESP32

Sebbene questi dati non facciano riferimento al periodo di raccolta dei dati, possono essere utilizzati per effettuare una stima del consumo energetico del dispositivo durante la fase di acquisizione. Mediamente, ESP32 impiega 2.08 secondi per connettersi al Wi-Fi, misurare umidità e temperatura, connettersi al broker MQTT, costruire il payload ed inviarlo a Raspberry Pi. Considerando una frequenza di acquisizione di un campione ogni 5 minuti, otteniamo un *duty cycle* pari allo 0.7%. Sono stati stimati consumi pari a 200mA per la fase attiva e pari a 20 μ A per la fase di deepsleep, il consumo medio risulta quindi pari a 0.86mA.

5. Machine Learning

In quest'ultima fase del progetto è stata progettata una pipeline di Machine Learning il cui scopo è la previsione dell'andamento della temperatura interna a partire dai dati raccolti dai sensori. Sono stati impiegati diversi modelli di ML in diversi scenari di previsione per cercare di capire se l'utilizzo del Machine Learning può essere applicato a questa casistica ed eventualmente, quali modelli si comportano meglio in questo scenario. Sono state utilizzate sei configurazioni sperimentali: le prime tre riguardano la previsione diretta della temperatura, all'istante immediatamente successivo, dopo 30 minuti e dopo 60 minuti, mentre le restanti tre riguardano la previsione del *delta T*, ovvero la variazione di temperatura dall'ultimo valore misurato, sugli stessi orizzonti temporali. Questo approccio può sembrare ridondante ma come verrà mostrato in seguito, sono stati ottenuti risultati interessanti.

5.1 Pre-processing dei dati

Prima della fase di addestramento, è stato effettuato un breve pre-processing dei dati raccolti. Inizialmente è stato effettuato un controllo su valori mancanti ed eventuali irregolarità nella sequenza di misurazioni. Sono poi state generate finestre mobili di lunghezza fissa (30 campioni per le tre grandezze misurate) per creare gli elementi su cui allenare i modelli. Nel caso delle previsioni dirette, il target è stato definito come la temperatura futura, ai vari intervalli di tempo, nel caso delle previsioni della variazione, il target è stato definito come la differenza tra la temperatura futura e l'ultima misurazione di temperatura presente all'interno delle finestre. Per i modelli classici, le finestre sono state convertite in vettori monodimensionali di 90 elementi mentre nel caso del modello LSTM, che assieme agli altri verrà descritto nel dettaglio nel paragrafo successivo, è stata mantenuta una struttura sequenziale. Per quanto riguarda la suddivisione del dataset nei due subset di training e di test, è stata utilizzata una divisione 80/20 mantenendo l'ordinamento cronologico dei dati, simulando un'effettiva previsione delle misure future. In questa fase di pre-processing sono state utilizzate le librerie *Pandas* e *NumPy*.

5.2 Modelli utilizzati

Per ognuna delle configurazioni della previsione descritte precedentemente, sono stati confrontati sette modelli. Il primo, non propriamente un modello di Machine Learning, è un modello *naive* di riferimento che, come previsione, utilizza l'ultimo valore di temperatura osservato nelle finestre. Come verrà descritto in seguito, per la tipologia di dati utilizzati, questo modello di riferimento rappresenta un benchmark piuttosto competitivo. Accanto a questo modello di base sono stati addestrati quattro modelli classici di regressione: *Linear Regression* (LR), *Ridge Regression* (RR), *Random Forest* (RF) e *Gradient Boosting* (GB), di scikit-learn. Sono poi stati impiegati due modelli neurali, *MultyLayer Perceptron* (MLP) e *Long Short-Term Memory* (LSTM), provenienti rispettivamente da scikit-learn e da TensorFlow / Keras.

5.3 Metriche utilizzate

Le prestazioni dei modelli appena descritti sono state valutate secondo tre metriche principali: *Mean Absolute Error* (MAE), *Root Mean Squared Error* (RMSE) e coefficiente di determinazione R^2 ($R2$). MAE misura l'errore medio assoluto tra i valori predetti e quelli reali. Anche RMSE misura l'errore medio ma attribuisce un peso maggiore ad errori di maggiore entità. Il coefficiente $R2$ è invece un indice che misura il legame tra la variabilità dei dati e la correttezza del modello predittivo: valori prossimi a 1 indicano un'elevata capacità predittiva, valori vicini allo 0 indicano prestazioni deboli mentre valori negativi descrivono un comportamento peggiore rispetto ad una previsione basata semplicemente sulla media.

5.4 Risultati

I risultati ottenuti, riportati in figura [6] e ordinati secondo le misure di RMSE ottenute, mostrano innanzitutto che, come ci si poteva aspettare, le prestazioni dei modelli tendono a peggiorare all'aumentare dell'orizzonte temporale di previsione. Questo comportamento risulta coerente con la natura del problema in quanto l'incertezza relativa alle dinamiche che causano variazioni di temperatura cresce progressivamente passando dalla previsione dopo 5 minuti, dopo 30 minuti e infine dopo 60 minuti.

Come accennato precedentemente, possiamo notare che le prestazioni della baseline Naive fornisce le prestazioni migliori nella previsione diretta della temperatura in tutti gli scenari considerati e in generale in quasi tutte le casistiche analizzate. Questo suggerisce che l'ambiente monitorato, ovvero la stanza dove sono state misurate le grandezze, presenta una notevole inerzia termica e che, di conseguenza, l'ultima osservazione disponibile rappresenta già di per sé una stima molto efficace del valore futuro.

I modelli lineari, in particolare LR e RR mostrano un comportamento complessivamente stabile e prestazioni molto vicine a quelle della baseline Naive. RF e GB si mantengono comunque su prestazioni competitive ma purtroppo non evidenziano nessun vantaggio sistematico rispetto ai modelli più semplici.

Per quanto riguarda i modelli neurali, LSTM presenta risultati migliori nella previsione dei *delta T* rispetto alla previsione diretta della temperatura. In particolare, il caso del *delta T* a 30 minuti rappresenta l'unico scenario, tra quelli analizzati, in cui un modello di Machine Learning presenta prestazioni migliori della baseline Naive. Questo può indicare che la sequenzialità dell'informazione può risultare più utile qualora il problema venga formulato in termini di variazioni termiche anziché di predizioni di valori assoluti.

Il modello MLP risulta il peggiore in tutti gli scenari analizzati, in particolare nella previsione diretta del valore successivo di temperatura dove mostra addirittura valori di $R2$ ampiamente negativi. Nelle varianti del problema con *delta T* presenta tuttavia un notevole miglioramento se rapportato alle prestazioni degli scenari precedenti, pur mantenendo performance peggiori rispetto a tutti gli altri modelli.

Osservando i risultati, è possibile, inoltre, notare che, generalmente, le controparti con *delta T* delle rispettive varianti a valore assoluto, mostrano risultati simili per i modelli che presentavano già prestazioni buone mentre mostrano notevoli miglioramenti nel caso di modelli, con prestazioni precedentemente scadenti, in particolare LSTM e MLP.

+5 min				+30 min				+60 min			
Model	MAE	RMSE	R2	Model	MAE	RMSE	R2	Model	MAE	RMSE	R2
Naive	0.034	0.065	0.993	Naive	0.115	0.174	0.949	Naive	0.173	0.247	0.894
LR	0.043	0.068	0.993	LR	0.124	0.183	0.944	LR	0.174	0.253	0.889
RR	0.045	0.069	0.992	RR	0.125	0.184	0.943	RR	0.174	0.253	0.888
RF	0.053	0.076	0.991	RF	0.137	0.194	0.937	GB	0.179	0.263	0.880
GB	0.055	0.078	0.990	GB	0.136	0.195	0.936	RF	0.184	0.266	0.877
LSTM	0.068	0.089	0.987	LSTM	0.149	0.212	0.925	LSTM	0.232	0.313	0.829
MLP	2.568	3.137	-14.978	MLP	2.564	3.166	-15.745	MLP	2.734	3.349	-18.530

deltaT +5 min				deltaT +30 min				deltaT +60 min			
Model	MAE	RMSE	R2	Model	MAE	RMSE	R2	Model	MAE	RMSE	R2
Naive	0.034	0.065	0.993	LSTM	0.121	0.172	0.950	Naive	0.173	0.247	0.894
RF	0.042	0.067	0.993	Naive	0.115	0.174	0.949	RR	0.174	0.252	0.889
RR	0.043	0.067	0.993	GB	0.125	0.178	0.947	LR	0.174	0.253	0.889
LR	0.043	0.068	0.993	RF	0.125	0.178	0.947	LSTM	0.182	0.254	0.888
GB	0.046	0.071	0.992	RR	0.124	0.182	0.944	GB	0.197	0.266	0.877
LSTM	0.050	0.073	0.991	LR	0.124	0.183	0.944	RF	0.197	0.268	0.875
MLP	0.079	0.110	0.980	MLP	0.172	0.243	0.901	MLP	0.199	0.295	0.848

Figura 6. Risultati dei test sui modelli ML

6. Conclusione

Il progetto descritto fino ad ora ha permesso di realizzare un sistema IoT distribuito in grado di monitorare grandezze ambientali, basato sull'integrazione di una ESP32, una Raspberry Pi e un'installazione di InfluxDB remota. Il sistema è stato in grado di acquisire, trasmettere e archiviare misurazioni di temperatura, umidità e luminosità rendendo possibile sia un'analisi statistica dei dati raccolti, sia la successiva applicazione di modelli di Machine Learning per la previsione dell'andamento termico dell'ambiente monitorato.

Eventuali sviluppi futuri comprendono l'estensione del periodo di raccolta dei dati, ottenendo un insieme di dati più ampio e variegato, nonché una frequenza di campionamento più bassa che potrebbe rendere più evidenti le variazioni significative delle grandezze misurate e, di conseguenza, favorire l'addestramento dei modelli predittivi. Per quanto riguarda l'infrastruttura hardware, potrebbe portare a risultati migliori un rafforzamento della stabilità della connessione Wi-Fi, accostata a meccanismi di memorizzazione locale dei dati in caso di disservizio. Ulteriori approfondimenti potrebbero infine riguardare l'introduzione di nuove variabili ambientali da misurare e l'impiego di modelli predittivi più avanzati. Infine, per ottimizzare i

consumi energetici del sistema, si potrebbe cercare di implementare un meccanismo di sincronizzazione che permetta a RaspberryPi di accendersi e mettersi in ascolto solamente quando necessario, senza dover restare in ascolto perenne.

7. Risorse

Tutti i file contenenti i codici utilizzati, i dataset e questa stessa relazione sono stati caricati presso:

<https://github.com/PietroMerlo/Progetto-IoT.git>